

Maîtriser les Licences Libres

En pratique

:::

Session 5 – Cours M1 Introduction aux Logiciels Libres

Objectifs de la séance

Ce que vous saurez faire

À la fin de cette séance, vous serez capables de :

- **Lire et interpréter** une licence courante (MIT, GPL, Apache)
- **Évaluer** la compatibilité entre licences
- **Choisir** une licence adaptée à un projet
- **Utiliser** les standards (SPDX) et outils
- **Comprendre** DCO, CLA et les mécanismes de contribution

Partie 1

Les licences permissives

Philosophie des licences permissives

■ « Faites ce que vous voulez avec ce code, tant que vous me créditez. »

Caractéristiques :

- Très peu de restrictions
- Pas d'obligation de partager les modifications
- Compatible avec le code propriétaire
- « Business-friendly »

Exemples : MIT, BSD (2 et 3 clauses), Apache 2.0, ISC

La licence MIT

La plus simple et la plus populaire.

MIT License

Copyright (c) [year] [fullname]

Permission is hereby granted, free of charge, to any person obtaining a copy of this software [...] to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software [...]

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND [...]

MIT : Analyse

Droits accordés

- Utiliser
- Copier
- Modifier
- Fusionner
- Publier
- Distribuer
- Sous-licencier
- Vendre

Obligations

- Conserver le copyright
- Conserver la licence

Limitations

- Aucune garantie
- Aucune responsabilité

Projets célèbres : jQuery, React, Rails, Node.js, .NET Core

La licence BSD : une histoire

BSD 4-clauses (Original, 1990)

- Conserver copyright dans source/binaire
- Clause « pas de pub avec mon nom »
- Clause « mentionner Berkeley dans la pub »

Problème : Incompatible GPL !

Projets célèbres : FreeBSD, OpenBSD, nginx

BSD 3-clauses (New BSD)

- Suppression clause de publicité
- Compatible GPL ✓

BSD 2-clauses (Simplified)

- Encore plus simple
- ≈ MIT

La licence Apache 2.0

Plus complète que MIT/BSD, avec protections supplémentaires.

Avantages

- Grant **explicite** de brevets
- Protection contre litiges brevets
- Définitions claires
- Fichier NOTICE requis

Obligations

- Conserver copyright/licence
- Documenter les modifications
- Inclure NOTICE si présent
- Mention de brevet si applicable

Projets célèbres : Android, Kubernetes, TensorFlow, Swift

Comparatif des licences permissives

Critère	MIT	BSD 3-cl	Apache 2.0
Longueur	~170 mots	~220 mots	~4500 mots
Attribution	Oui	Oui	Oui
Protection brevets	Non	Non	Oui
Fichier NOTICE	Non	Non	Oui
Clause « pas de pub »	Non	Oui	Non
Complexité	Très faible	Faible	Moyenne

Partie 2

Les licences copyleft

Philosophie du copyleft

« Vous pouvez utiliser ce code librement, mais vos modifications doivent rester libres. »

Caractéristiques :

- Les libertés sont **virales**
- Les œuvres dérivées doivent garder la même licence
- Empêche la « privatisation » du code
- Plus restrictif pour l'usage commercial mixte

Œuvre dérivée vs œuvre collective

Question centrale pour comprendre le copyleft :

Œuvre dérivée

- Modification, adaptation, transformation
- L'œuvre originale est « incorporée »
- Le copyleft s'applique à l'ensemble
- Ex : fork, modification du code

Œuvre collective (agrégation)

- Assemblage d'œuvres indépendantes
- Chaque partie reste distincte
- Le copyleft ne « contamine » pas
- Ex : distribution Linux (noyau + apps)

Copyleft fort vs faible

Copyleft fort (GPL, AGPL)

- Tout le programme dérivé doit être libre
- Si vous linkez avec du GPL, tout devient GPL
- « Contaminant » (terme péjoratif)

Copyleft faible (LGPL, MPL)

- Seules les modifications de la lib sont concernées
- Vous pouvez linker avec du code propriétaire
- La lib reste libre, pas l'ensemble

La licence GPL

La licence copyleft la plus connue, créée par Richard Stallman.

Versions :

- **GPL v1** (1989) – Version initiale
- **GPL v2** (1991) – Plus précise, très utilisée (Linux kernel)
- **GPL v3** (2007) – Brevets, DRM, Tivoization

GPL : Droits et conditions

Droits accordés

- Utiliser pour tout usage
- Étudier le code source
- Modifier le logiciel
- Redistribuer des copies
- Distribuer des versions modifiées

Conditions

- Inclure le code source
- Conserver la licence GPL
- Conserver les notices de copyright
- Ne pas imposer de restrictions supplémentaires

Si vous distribuez un logiciel basé sur du GPL, vous devez distribuer tout le code source sous GPL.

GPL v2 vs GPL v3

Aspect	GPL v2	GPL v3
Brevets	Implicite	Grant explicite
DRM	Non traité	Anti-DRM (anti-Tivoization)
Compatibilité	Moins compatible	Compatible Apache 2.0
Longueur	~2500 mots	~5500 mots
Usage	Linux kernel	Projets GNU

Tivoization : TiVo utilisait Linux (GPL v2) mais empêchait les utilisateurs de modifier le logiciel sur leur appareil. GPL v3 interdit cela.

La licence AGPL (Affero GPL)

Problème : La GPL ne s'applique que lors de la **distribution**. Un service SaaS n'est pas distribué → pas d'obligation de partager le code.

Solution AGPL :

- Si vous offrez le logiciel comme service réseau
- Vous devez fournir le code source aux utilisateurs
- Même sans distribution de binaires

Projets célèbres : Mastodon, Nextcloud, Grafana, Bluemind

La licence LGPL (Lesser GPL)

Copyleft **faible** – conçue pour les bibliothèques.

Ce qui reste libre

- La bibliothèque elle-même
- Les modifications de la bibliothèque

Ce qui peut être propriétaire

- Le programme qui utilise la bibliothèque
- À condition de lier dynamiquement

Projets célèbres : glibc, Qt (partiellement), GTK

Le débat du linking

Question : Un programme qui utilise une bibliothèque GPL est-il une œuvre dérivée ?

Type de linking	Position FSF	Position alternative
Static linking	Œuvre dérivée	Consensus
Dynamic linking	Œuvre dérivée	Œuvre collective ?
Import (Python, JS ...)	Œuvre dérivée	Très débattu
Appel réseau/API	Non dérivée	Consensus

En pratique : Pas de jurisprudence claire. La prudence recommande de suivre l'interprétation de l'auteur.

La licence MPL (Mozilla Public License)

Copyleft **au niveau du fichier**.

Principe

- Les fichiers modifiés restent sous MPL
- Les nouveaux fichiers peuvent être propriétaires
- Plus permissif que LGPL

Avantages

- Compatible avec Apache 2.0
- Clair sur le périmètre
- Bon compromis

Projets célèbres : Firefox, Thunderbird, LibreOffice (partiellement)

Partie 3

Compatibilité des licences

Le problème de la compatibilité

Quand vous combinez du code sous différentes licences, les obligations se **cumulent**.

Questions clés :

- Puis-je utiliser du code sous licence X dans un projet sous licence Y ?
- Sous quelle licence puis-je redistribuer le résultat ?

Règle générale

Permissif → Copyleft : OK

- MIT → GPL ✓
- BSD → GPL ✓
- Apache 2.0 → GPL v3 ✓

Le code permissif peut être intégré dans du copyleft.

Copyleft → Permissif : NON

- GPL → MIT ✗
- GPL → propriétaire ✗

Le copyleft « contamine » : tout doit rester sous la même licence.

Matrice de compatibilité simplifiée

Source ↓ / Dest →	MIT	Apache 2.0	GPL v2	GPL v3	LGPL	AGPL
MIT	✓	✓	✓	✓	✓	✓
Apache 2.0	✓	✓	✗	✓	✓*	✓
GPL v2	✗	✗	✓	✗*	✗	✗
GPL v3	✗	✗	✗	✓	✗	✓
LGPL	✗	✗	✓	✓	✓	✓
AGPL	✗	✗	✗	✗	✗	✓

* GPL v2 « only » incompatible avec GPL v3. Apache 2.0 compatible avec LGPL v3, pas LGPL v2.1. « GPL v2 or later » compatible.

Conseils pratiques

1. **Vérifiez toujours** la licence avant d'intégrer du code externe
2. **Documentez** les licences de toutes vos dépendances
3. **Évitez** de mélanger GPL v2 only avec autre chose
4. **Préférez** des licences avec clause « or later » pour la flexibilité
5. En cas de doute, **consultez** un expert ou utilisez un outil

Partie 4

Choisir une licence

Règle d'or : ne créez pas votre propre licence !

Conseil essentiel : Utilisez toujours une licence existante et reconnue.

Pourquoi éviter les licences « maison » ? Choisissez une licence populaire

- Risques juridiques (texte non testé)
- Problèmes d'incompatibilité
- Confusion pour les utilisateurs
- Travail de rédaction complexe
- MIT, BSD, Apache 2.0 (permissives)
- GPL, LGPL, AGPL (copyleft)
- MPL 2.0 (copyleft fichier)
- Ou celle de votre écosystème

■ « Il y a déjà trop de licences. N'en ajoutez pas. » – OSI

Questions à se poser

Avant de choisir une licence, réfléchissez à :

1. **Objectif** : Que voulez-vous accomplir ?
2. **Communauté** : Qui va contribuer ? Des entreprises ?
3. **Écosystème** : Quelles licences utilisent les projets similaires ?
4. **Compatibilité** : Avec quoi votre code sera-t-il combiné ?
5. **Business model** : Comment comptez-vous monétiser (si applicable) ?

Guide de choix simplifié (1/2)

Choisissez MIT/Apache si :

- Vous voulez une adoption maximale
- Les entreprises doivent pouvoir l'utiliser
- Vous ne vous souciez pas des modifications non partagées

Choisissez GPL si :

- Vous voulez garantir que le code reste libre
- Vous voulez éviter la « privatisation »
- Vous visez une communauté militante

Guide de choix (2/2)

Choisissez LGPL si :

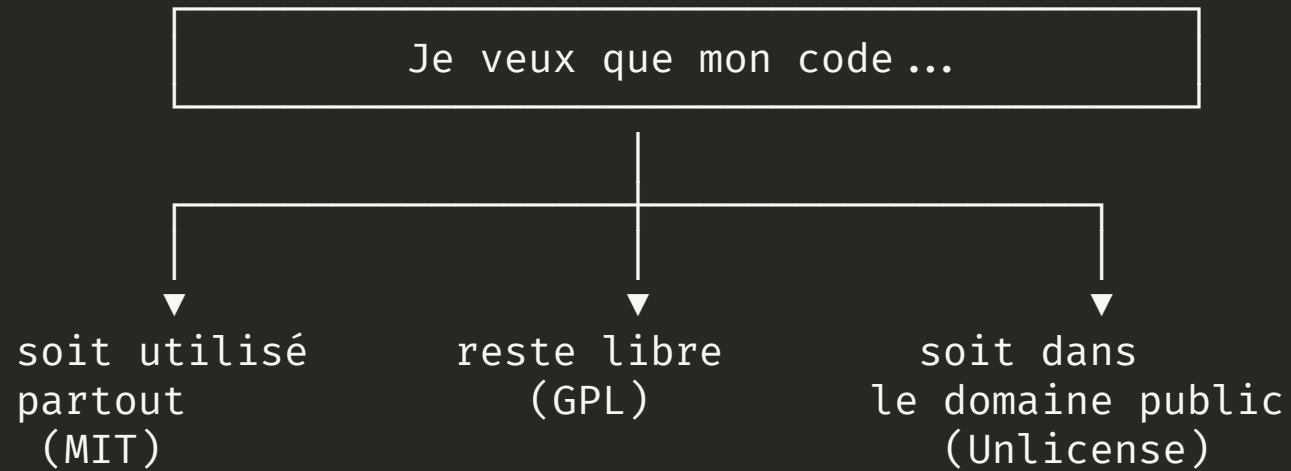
- Vous développez une bibliothèque
- Vous voulez permettre l'usage propriétaire
- Mais garder les améliorations libres

Choisissez AGPL si :

- Votre logiciel est un service web
- Vous voulez empêcher l'exploitation SaaS sans contribution
- Forte conviction éthique

Ressource : choosealicense.com

GitHub propose un guide simple (simpliste?) : <https://choosealicense.com>



Partie 5

Standards et outils

SPDX : Software Package Data Exchange

Standard pour identifier et décrire les licences.

Identifiants SPDX :

- MIT
- Apache-2.0
- GPL-3.0-only
- GPL-3.0-or-later
- LGPL-2.1-only

Usage : Dans `pyproject.toml`, `package.json`, `Cargo.toml`, `pom.xml`, headers de fichiers ...

Expressions SPDX

Pour les cas complexes (multi-licences) :

```
GPL-2.0-only OR MIT           # Au choix  
Apache-2.0 AND MIT           # Les deux s'appliquent  
GPL-2.0-or-later WITH Classpath-exception-2.0 # Avec exception
```

Exemple concret :

```
// package.json  
{  
  "license": "(MIT OR Apache-2.0)"  
}
```

Headers de fichiers

Bonne pratique : Ajouter un header de licence dans chaque fichier source.

```
\# SPDX-License-Identifier: MIT  
\# Copyright (c) 2024 Your Name  
\#  
\# This file is part of MyProject.
```

Format SPDX recommandé :

```
// SPDX-License-Identifier: GPL-3.0-or-later  
// SPDX-FileCopyrightText: 2024 Author Name <email>
```

Outils de détection de licence

Outil	Type	Usage
REUSE	CLI	Détection automatique + compliance
licensee	CLI	Détection automatique
ScanCode	CLI/Lib	Analyse complète
FOSSA	SaaS	Compliance entreprise
Snyk	SaaS	Sécurité + licences
GitHub	Web	Détection intégrée
FOSSology	Self-hosted	Audit complet

Ces outils seront étudiés plus en détail dans une séance ultérieure.

Partie 6

Contribuer à un projet

Qui détient le copyright sur les contributions ?

Par défaut : **le contributeur** garde le copyright sur son code.

Conséquences :

- Le projet a des dizaines/centaines de détenteurs de droits
- Impossible de changer de licence sans l'accord de tous
- Difficile de poursuivre les violations
- Identification floue des contributeurs et de leurs droits

Solutions possibles :

- DCO (Developer Certificate of Origin)
- CLA (Contributor License Agreement)
- Copyright Assignment

DCO : Developer Certificate of Origin

Déclaration légère du contributeur :

```
Developer Certificate of Origin  
Version 1.1
```

```
By making a contribution to this project, I certify that:  
(a) The contribution was created in whole or in part by me and I  
    have the right to submit it under the open source license  
    indicated in the file; or  
(b) [...]
```

Utilisation : Ligne **Signed-off-by:** dans le commit.

```
git commit -s -m "Fix bug in parser"  
\# Ajoute: Signed-off-by: Your Name <your@email.com>
```


CLA : Contributor License Agreement

Le contributeur **conserve son copyright** mais accorde une **licence large** au projet.

Avantages

- Licence perpétuelle, irrévocable
- Permet le relicensing
- Facilite les poursuites en cas de violation
- Sensibilise aux questions de PI

Inconvénients

- Friction pour les contributeurs
- Souvent perçu négativement
- Paperasse administrative

Note : Contrairement au Copyright Assignment, le contributeur **conserve sa propriété**.

Exemples : Apache CLA, Google CLA, Microsoft CLA

Copyright Assignment (cession)

Le contributeur **transfère la propriété** de son copyright au projet.

Caractéristiques

- Transfert total de propriété
- Le contributeur n'est plus titulaire
- Nécessite parfois un « license back »

Utilisé par

- FSF (projets GNU)
- Rare ailleurs (la plupart des fondations utilisent des CLA)

Différence clé avec CLA : Le CLA accorde une **licence**, l'Assignment transfère la **propriété**.

Comparatif

Mécanisme	Propriété conservée	Relicensing
Rien	Oui	Impossible
CLA	Oui (licence accordée)	Possible*
Assignment	Non (transfert)	Possible

* Si le CLA le permet.

GitHub et « inbound=outbound »

Extrait des Terms of Service de GitHub :

« Whenever you add Content to a repository containing notice of a license, **you license that Content under the same terms.** »

En pratique : sur GitHub, contribuer = accepter la licence du projet.

Partie 7

Cas spéciaux

Dual Licensing

Un projet disponible sous **deux licences** au choix.

Modèle classique :

- GPL pour l'usage libre
- Licence commerciale pour l'usage propriétaire

Exemples

- MySQL (GPL + Commercial)
- Qt (LGPL + Commercial)

Avantages éditeur

- Revenu commercial
- Communauté libre

Exceptions de licence

Ajout de permissions spéciales à une licence existante.

Exemples courants :

- **Classpath Exception** (Java) : Permet de linker avec du code propriétaire
- **GCC Runtime Exception** : Les programmes compilés ne sont pas « contaminés »
- **Font Exception** : Les documents utilisant une police libre ne sont pas GPL

Le domaine public

Domaine public « naturel »

- Copyright expiré (70 ans après la mort)
- Rare pour le logiciel

Dédication volontaire

- CC0 (Creative Commons Zero)
- Unlicense
- WTFPL

Problème : Dans certaines juridictions (France, Allemagne), on ne peut pas vraiment renoncer au copyright. D'où CC0 qui accorde une licence très permissive en fallback.

Creative Commons

Licences pour le **contenu** (texte, images, musique), pas le code.

Variante	Signification	Libre ?
CC0	Domaine public	Oui
CC BY	Attribution	Oui
CC BY-SA	Attribution + ShareAlike	Oui
CC BY-NC	Non-Commercial	Non
CC BY-ND	No Derivatives	Non

Usage : Documentation, assets, datasets. Pas recommandé pour le code.

Licences « Source Available »

Ni libre, ni propriétaire traditionnel.

Exemples récents

- **SSPL** (MongoDB)
- **BSL** (HashiCorp, MariaDB)
- **Elastic License**
- **Commons Clause**

Pourquoi ?

- Protection contre cloud providers
- « AWS-proofing »
- Monétisation directe

Important : Ces licences ne sont **pas** open source selon l'OSI.

Changement de licence (Relicensing)

Peut-on changer la licence d'un projet ?

Oui si :

- Vous détenez 100% des droits (ex: auteur unique, auteurs salariés de la même organisation, cession de propriété par les contributeurs, CLA qui le permet)
- Ou : passage vers licence compatible (ex: MIT → GPL)

Non si :

- Contributeurs multiples sans leur accord explicite
- Code tiers sous copyleft incorporé

Question clé : Contrôlez-vous les droits sur **tout** le code ?

Cas célèbres de relicensing

Projet	Avant	Après	Raison
MongoDB	AGPL	SSPL	Anti-cloud
Elastic	Apache 2.0	SSPL/ELv2	Anti-AWS
HashiCorp	MPL	BSL	Monétisation
Redis modules	BSD	Source available	Idem

Réactions : Forks communautaires (OpenSearch, OpenTofu, Valkey ...)

Étude de cas : Chardet (2026)

Les faits :

- **chardet** : bibliothèque Python de détection d'encodage (~130M téléchargements/mois)
- Maintenu depuis 12 ans par Dan Blanchard, auteur original : Mark Pilgrim
- **Version 7.0** : réimplémentée par IA (Claude), 48× plus rapide, multi-cœurs
- Licence changée de **LGPL** → **MIT**
- Méthode : API + tests fournis à l'IA, sans lire le code source
- Similarité JPlag : <1.3% avec les versions précédentes

Chardet : la controverse

Position du mainteneur

- Réimplémentation « clean room » via IA
- Code entièrement nouveau
- Pas d'obligation de conserver la LGPL
- Amélioration majeure (performance)

Objection de l'auteur original

- Exposition massive au projet existant
- Contrat social avec les contributeurs
- La LGPL protégeait leurs contributions
- Relicensing = rupture de confiance

Question centrale : Une réimplémentation par IA est-elle une « œuvre nouvelle » ou une « œuvre dérivée » ?

Chardet : trois points de vue

Armin Ronacher (créateur de Flask) – *favorable* :

- « La GPL va contre l'esprit du partage en restreignant ce qu'on peut faire. »
- Voit l'IA comme libératrice des contraintes copyleft

Antirez (créateur de Redis) – *favorable* :

- « GNU a réimplémenté UNIX, Linux a réimplémenté via Minix. L'IA accélère un processus qui a toujours existé. »
- Le copyright protège l'expression, pas les idées

Zoë Kooyman (FSF) – *opposée* :

- « Refuser aux autres les droits que vous avez reçus est profondément antisocial, quelle que soit la méthode. »

Chardet : les enjeux

Arguments « pro-réimplémentation »

- Légalement permis (pas de copie)
- Précédent historique (GNU, Linux)
- Accélère l'innovation
- Rééquilibre face aux grandes entreprises

Arguments « anti-réimplémentation »

- Légal ≠ légitime
- Direction inversée (copyleft → permissif)
- Contrat social brisé
- Érode les protections du copyleft

Ironie relevée : Vercel a réimplémenté Bash (GPL) en MIT, mais s'est offusqué quand Cloudflare a réimplémenté Next.js (MIT).

Chardet : questions ouvertes

1. **Le droit d'auteur protège-t-il les spécifications ?** (API, tests, comportement)
2. **Le code généré par IA est-il protégeable ?** (Pas de jurisprudence claire)
3. **Faut-il un « copyleft de spécification » ?** (TGPL proposé par certains)
4. **La vitesse change-t-elle la nature du processus ?** (Ce qui prenait 5 ans prend 5 jours)

« Nous sommes dans un nouveau monde et nous avons très peu d'idées sur comment y naviguer. » – Armin Ronacher

Chardet : références

Sources et analyses :

- [Discussion GitHub originale](<https://github.com/chardet/chardet/issues/327#issuecomment-4005195078>)
- [The Ship of Theseus](#) – Armin Ronacher
- [AI and OSS licensing](#) – Antirez (Salvatore Sanfilippo)
- [Legal vs Legitimate](<https://writings.hongminhee.org/2026/03/legal-vs-legitimate/>) – Hong Minhee

Résumé



Ce qu'il faut retenir

1. **Permissif** (MIT, Apache) → peu de contraintes, adoption maximale
2. **Copyleft** (GPL) → garantit que le code reste libre
3. **Copyleft faible** (LGPL, MPL) → compromis pour bibliothèques
4. **Compatibilité** : Permissif → Copyleft OK, inverse NON
5. **SPDX** : Standard pour identifier les licences
6. **DCO/CLA** : Mécanismes pour gérer les contributions
7. **Ne créez jamais votre propre licence !**

Tableau récapitulatif

Licence	Type	Copyleft	Brevets	Usage typique
MIT	Permissive	Non	Non	Libs, frameworks
Apache 2.0	Permissive	Non	Oui	Corporate, cloud
GPL v2	Copyleft fort	Oui	Implicite	Linux kernel
GPL v3	Copyleft fort	Oui	Oui	GNU tools
AGPL	Copyleft + SaaS	Oui	Oui	Services web
LGPL	Copyleft faible	Partiel	Oui	Bibliothèques
MPL 2.0	Copyleft fichier	Partiel	Oui	Mozilla

Checklist avant de publier un projet

- ☐ Fichier LICENSE à la racine
- ☐ Headers SPDX dans les fichiers source
- ☐ SPDX identifier dans package.json/Cargo.toml/etc.
- ☐ Fichier CONTRIBUTING expliquant DCO/CLA si applicable
- ☐ Liste des dépendances et leurs licences vérifiée
- ☐ Compatibilité des licences validée

Pour la prochaine séance

Séance 6 : Contribuer à un projet open source

Préparation suggérée :

- Créer un compte GitHub si pas déjà fait
- Explorer un projet open source qui vous intéresse
- Lire le fichier CONTRIBUTING d'un projet populaire

Questions ?

⋮

Passons au TD